

Phase 1 - Frontend Requirement Document

1 Project Overview

Project Name: UST Employee Management (JIRA-lite)

Phase: 1 – Frontend & UI

Objective:

- Build a **React-based frontend** to manage employee tasks
- Connect to **pre-built FastAPI backend** via Axios
- Learn **CRUD operations, status workflow, and conditional rendering**
- Beginner-friendly, minimal complexity, production-like

Consider: Backend is **already provided**. You do **not need to write backend code**.

2 Roles

Role	Permissions in Phase 1
Admin	View all employees, create new employee, delete employee
Manager	Assign tasks, update status of tasks they created
Employee	View assigned tasks, update task status

Phase 1 focuses on React UI & API consumption. Role-based logic can be simulated via dropdowns or buttons.

3 Task Entity Schema

Field	Type	Example	Notes
<code>id</code>	int	101	Primary key

Field	Type	Example	Notes
<code>title</code>	string	"Update client database"	Short task description
<code>description</code>	string	"Update MySQL database for client XYZ"	Detailed info
<code>status</code>	string	"TO_DO"	TO_DO / IN_PROGRESS / REVIEW / DONE
<code>priority</code>	string	"High"	High / Medium / Low
<code>assignee</code>	string	"Rahul Sharma"	Employee assigned
<code>department</code>	string	"Digital"	Department task belongs to
<code>createdAt</code>	date/time	"2025-12-15"	Auto-filled
<code>createdBy</code>	string	"Suresh Kumar"	Employee who created task
<code>updatedAt</code>	date/time	"2025-12-16"	Auto-updated on change
<code>updatedBy</code>	string	"Rahul Sharma"	Employee who last updated task

Task Status Flow:

TO_DO → IN_PROGRESS → REVIEW → DONE

- Status updates should be handled via **dropdown/buttons** in the UI
- Display **createdBy, updatedBy, createdAt, updatedAt** in table view

4 Database Overview

Database Name: `ust_employee_db`

Tables :

1. employees

- `id` (int, PK)
- `name` (varchar)
- `email` (varchar)

- `designation` (varchar)
- `department` (varchar)
- `location` (varchar)
- `status` (varchar: Active / Inactive / On Probation)
- `salary` (float)
- `date_of_joining` (date)

2. departments

- `id` (int, PK)
- `name` (varchar)
- `manager_id` (FK to employees.id)

Backend handles all DB operations. Front End only consume APIs via Axios.

5 Git / Project Structure

Suggested Frontend Directory:

```
ust-employee-management/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── EmployeeForm.jsx
│   │   │   ├── EmployeeList.jsx
│   │   │   └── TaskRow.jsx
│   │   ├── services/
│   │   │   └── employeeService.js
│   │   └── App.jsx
│   ├── index.html
│   └── package.json
└── README.md
```

Naming Conventions:

- Components: PascalCase (`EmployeeForm.jsx`)
- Variables / state: camelCase (`employeeData`)
- Service files: `employeeService.js`
- Git branches: `phase-1-ui`

6 API Endpoints (Pre-built Backend)

Operation	Endpoint	Method	Payload	Response
Get all employees	<code>/api/employees</code>	GET	-	Array of employee JSON
Get single employee	<code>/api/employees/:id</code>	GET	-	Employee JSON
Create employee	<code>/api/employees</code>	POST	Employee JSON	<code>{ "id": 101, "message": "Employee created successfully" }</code>
Update employee	<code>/api/employees/:id</code>	PUT	Employee JSON	<code>{ "message": "Employee updated successfully" }</code>
Delete employee	<code>/api/employees/:id</code>	DELETE	-	<code>{ "message": "Employee deleted successfully" }</code>

Sample JSON Payload (Create / Update):

```
{
  "name": "Rahul Sharma",
  "email": "rahul.sharma@ust.com",
  "designation": "Software Engineer",
  "department": "Digital",
  "location": "Bangalore",
  "status": "Active",
  "salary": 75000,
  "date_of_joining": "2022-06-15"
}
```

7 Frontend Requirements (Phase 1)

Components to build:

1. **EmployeeForm.jsx**

- Input fields for all employee details
- Submit button calls `employeeService.createEmployee()` or `updateEmployee()`
- Reset form after submission

2. **EmployeeList.jsx**

- Display all employees in a table
- Columns: Name, Email, Designation, Department, Location, Status, Salary, CreatedBy, UpdatedBy, CreatedAt, UpdatedAt
- Action buttons: Edit / Delete

3. **TaskRow.jsx** (Optional for status workflow demonstration)

- Dropdown to update status (TO_DO → IN_PROGRESS → REVIEW → DONE)
- Show last updated by

4. **Services / API Calls** (`employeeService.js`)

- Use Axios for all GET / POST / PUT / DELETE calls
-

8 Sample Data (100+ Employees)

- Departments: Digital, HR, Sales, IT Support
 - Locations: Bangalore, Mumbai, Hyderabad, Chennai, Pune
 - Roles: Software Engineer, Manager, Analyst, Tester
 - Status: Active, Inactive, On Probation
-

9 Submission Guidelines

- Push code to Git branch: `phase-1-ui`
- Include **screenshots of working CRUD operations**

- Optional: Tailwind styling for UI
 - Deadline: [Insert date]
-

10 Notes

- **Focus on learning:** Axios CRUD + React UI
 - **Phase 1 scope:** Frontend only, backend is pre-built
 - **Audit fields** (`createdBy` , `updatedBy` , `createdAt` , `updatedAt`) must be displayed
 - **Status workflow:** TO_DO → IN_PROGRESS → REVIEW → DONE
-